Terraform State File Cheatsheet for Beginners

The state file is Terraform's memory card that maps your code to real cloud resources. Understanding how to view and manage state keeps your cloud setup safe and error-free.

What you'll learn:

1. Viewing All Managed Resources
2. Inspecting Resource Details
3. Importing Existing Infrastructure
4. Storing State Safely in Remote Cloud

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Viewing All Managed Resources

The `terraform state list` command gives you a clean list of every cloud resource recorded in state. It works like an inventory list so you can see what Terraform tracks before making changes.

```
$terraform state list
```

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Inspecting Resource Details

Use `terraform state show` with a resource address to read its exact properties. This displays attributes like IDs, IP addresses, and tags saved in your state file.

```
$terraform state show aws_instance.web_server
```

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Importing Existing Infrastructure

When a resource already exists in AWS but is missing from Terraform state, use import. First define an empty resource block in your code, then run import with the cloud ID.

```
$terraform import aws_s3_bucket.app_data my-app-data-bucket-2026
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Storing State Safely in Remote Cloud

Save your state file in AWS S3 instead of your personal laptop so teammates can collaborate. Adding a DynamoDB table enables file locking to prevent simultaneous edits.

```
terraform {
  backend "s3" {
    bucket = "company-tf-state-bucket"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
  }
}
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always use a remote backend like AWS S3 or Azure Blob Storage for team projects.
+ Enable DynamoDB state locking to stop two people from running apply at the same time.
+ Enable bucket versioning on your remote backend to recover from accidental state corruption.
+ Run `terraform state list` to check tracked resources before making major code updates.
+ Keep sensitive outputs secure by marking them as sensitive in your Terraform code.

Watch Out For

! Manually editing `terraform.tfstate` JSON files using a standard text editor.
! Committing `terraform.tfstate` files containing passwords to public GitHub repositories.
! Deleting state files locally expecting Terraform to auto-detect active cloud resources.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh